Praktisches Requirements Engineering

Samuel Fricker, 2025-09-14

Kapitel 1: Einführung in Requirements Engineering

- In diesem Kapitel lernen wir die Theorie des Requirements Engineering kennen. Wir befassen uns mit den Grundlagen, der Motivation und dem Anwendungsbereich von Requirements Engineering.
- Im Requirements Engineering geht es darum, dass ein Entwicklungsteam gemeinsam mit den Stakeholdern ein Produkt nach den Prinzipien von Co-Design gemeinsam gestalten kann.
- Wichtigkeit von Requirements Engineering verstehen,
  Zentrale Konzepte des Requirements Engineerings kennen,
  Vorgehen für Requirements Engineering im Informatikprojekt kennen.
- Ideen, Bedürfnisse, Ziele, Anforderungen und Rahmenbedingungen

1.1 Grundlagen des Requirements Engineering

Dieser Abschnitt beschreibt die grundlegenden Ideen und Konzepte des Requirements Engineering für das gemeinsame Design eines Softwareprodukts, bei dem sowohl das Entwicklungsteam als auch die Stakeholder beteiligt sind. Dabei werden die Herausforderungen, die Ziele, die Aktivitäten und die verwendeten Konzepte vorgestellt. Zum Abschluss wird aufgezeigt, wie Requirements Engineering in unterschiedlichen Vorgehensmodellen der Softwareentwicklung angewendet wird – von agilen, iterativen Ansätzen bis hin zu sequenziellen Wasserfallmodellen.

Software wird nicht allein durch die guten Absichten der Entwickelnden erfolgreich. Damit Software erfolgreich ist, muss sie akzeptiert werden und den gewünschten Nutzen bringen.

Fallstudie *Skyguide Atmas*

Es lohnt sich, den Business Case von Requirements Engineering zu verstehen. Studiere dazu bitte den folgenden Pressebericht, der im *Tages-Anzeiger* erschienen ist. Der Bericht behandelt die Entwicklung eines neuen Flugsteuerungssystems für Skyguide, das von der Firma Thales geliefert wird und den Namen *Atmas* trägt.



In einer trockenen Medienmitteilung gab Skyguide im Juli letzten Jahres das Ende des Projekts "Atmas" bekannt. Das vorgeschlagene System für die Flugsicherung sei "ökonomisch wenig sinnvoll" gewesen. "Heute sagen die Politiker, das System sei von Anfang an veraltet gewesen. Als das Projekt abgebrochen wurde, waren jedoch bereits 55 Millionen CHF (rund 55 Mio. €) ausgegeben worden. Ende Juni wird Skyguide erklären, wie die teure Fehlinvestition zustande kam.

Politiker haben den Fall Skyguide bereits in den letzten Tagen untersucht. Der "Tages-Anzeiger" erhielt Informationen aus einer Sitzung mit Mitgliedern der Geschäftsprüfungs- und Finanzkommission des Parlaments. Die Informationen zeigen, dass der Verwaltungsrat von Skyguide, die für die Überwachung von Atmas zuständigen Mitarbeiter des Eidgenössischen Verkehrsdepartements (UVEK) und die involvierten Mitarbeiter des Verteidigungsdepartements (VBS) zu spät reagiert haben.

Das UVEK hat erst im Nachhinein einen Bericht über die Katastrophe angefordert. Dieser Bericht zeigt, dass der Verwaltungsrat von Skyguide das teure Projekt kaum überwacht hat. Auch bestand die Projektgruppe aus vielen Ingenieuren, aber nicht aus zukünftigen Nutzern. Schliesslich wurden zu wenige Tests durchgeführt, um die Versprechen des Atmas-Herstellers Thales zu überprüfen.

Bundesrat Moritz Leuenberger sagte an der Sitzung, der neue Verwaltungsratspräsident Guy Emmenegger habe ihn im letzten Sommer mit folgenden Worten gewarnt: "Atmas ist ein Fass ohne Boden – so kann es nicht weitergehen." Nach mehreren Sitzungen mit dem Verwaltungsrat wurde der Stopp des Projekts beschlossen. "Wir wollen jetzt die Lehren aus dem Fall ziehen", sagt Hans Werder, Generalsekretär des UVEK. 25 der 55 Millionen CHF gelten als verloren und sind abgeschrieben. Der Rest wurde von Skyguide als "wertvolles Know-how" verbucht.

Beantworte anschliessend die folgenden Fragen:

- War *Atmas* erfolgreich?
- Was waren die drei wichtigsten Gründe?

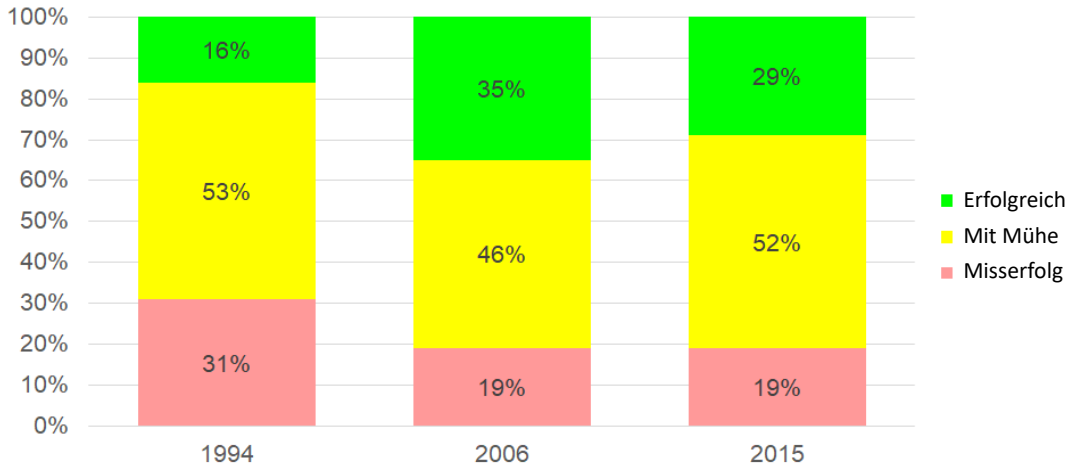


Abbildung 1: Erfolgsquoten von Softwareporjekten in den Jahren 1994, 2006 und 20215 nach Angaben der Standish Group.

Abbildung 1 zeigt die Erfolgsquoten von Softwareprojekten in den Jahren 1994 [Clancy 1995], 2006 und 2015. Diese stammen aus der Umfragereihe **„CHAOS Reports“**, in der zahlreiche Chief Information Officers (CIOs) in den USA befragt wurden.

- **Erfolg** bedeutet: Das Projekt wurde rechtzeitig, im Budgetrahmen und mit allen vereinbarten Funktionen abgeschlossen.
- **Herausgefordert** bedeutet: Das Projekt wurde zwar abgeschlossen, aber mit Budgetüberschreitung, Verzögerung oder reduziertem Funktionsumfang.
- **Fehlschlag** bedeutet: Das Projekt wurde abgebrochen, bevor es veröffentlicht werden konnte.

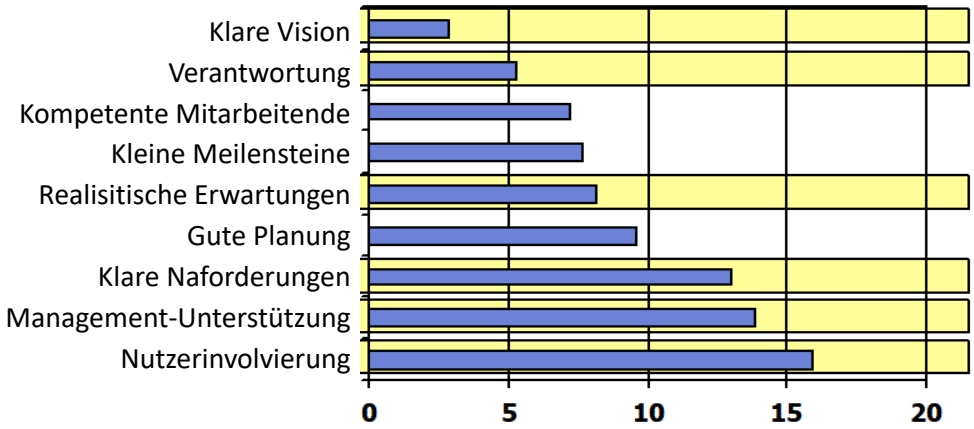


Abbildung 2: Die wichtigsten Erfolgsfaktoren für Software-Projekte im Jahr 1995, nach Angaben der Standish Group. Die Länge der Balken zeigt den Prozentsatz der Antworten, die den in der Zeile angegebenen Erfolgsfaktor bestätigen.

Ein interessantes Detail ist der grosse Sprung bei den Erfolgsquoten zwischen 1994 und 2006. Danach haben sich die Erfolgsquoten gemäss den Daten kaum mehr verändert. Die befragten CIOs nannten die in **Abbildung 2** gezeigten Erfolgsfaktoren.

Die gelb markierten Faktoren in Abbildung 2 beziehen sich auf Praktiken des Requirements Engineering. Nicht markierte Faktoren stammen aus dem Projektmanagement. Der deutliche Anteil von RE-Praktiken unter den Erfolgsfaktoren zeigt: Gemäss dieser Studie ist **Requirements Engineering die wichtigste Disziplin für erfolgreiche Software**. Durch die Berücksichtigung dieser Faktoren stiegen die Erfolgsquoten bis 2006 deutlich an.

Die CHAOS Reports zeigen, dass Requirements Engineering entscheidend für den Projekterfolg ist. Die Softwareindustrie konnte nach 1995 grosse Verbesserungen erzielen, indem sie sich auf folgende Schlüsselfaktoren konzentrierte:

- Einbezug der Nutzenden,
- Unterstützung durch das Management,
- klare Anforderungen.

Damit trägt Requirements Engineering dazu bei, dass das Projektmanagement Software **planen und liefern kann – rechtzeitig, im Budgetrahmen und mit den erwarteten Funktionen**.

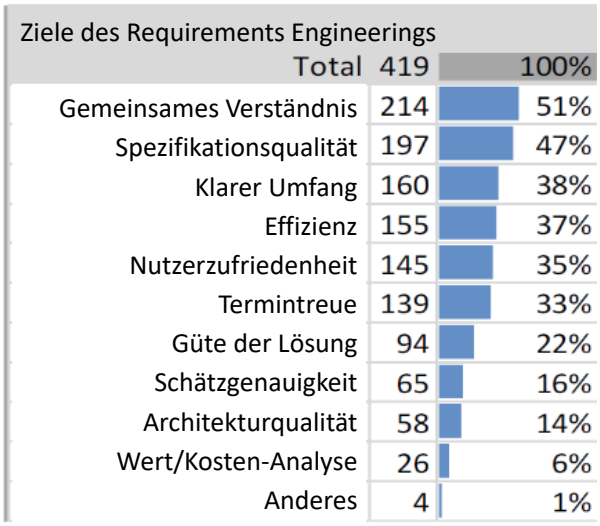


Abbildung 3: Die häufigsten Ziele des Requirements Engineering [Fricker 2015].

Im Jahr 2015 befragte ich 419 Organisationen zu den Zielen, die sie mit Requirements Engineering in ihrem letzten Projekt verfolgten. **Abbildung 3** zeigt die Ergebnisse [Fricker 2015].

Die beiden häufigsten Ziele bestätigten die Ergebnisse des CHAOS Reports von 1995:

1. **Ein gemeinsames Verständnis erreichen**

2. **Eine qualitativ hochwertige Anforderungsspezifikation erstellen**

- *Gemeinsames Verständnis* bedeutet: Nutzende, Management und weitere Stakeholder sind so stark eingebunden, dass das Entwicklungsteam dieselben Begriffe verwendet und die Stakeholder überzeugt sind, dass die geplante Software in den Zielkontext passt und für die angestrebten Ziele nützlich ist.
- *Qualität der Spezifikation* bedeutet: Die Anforderungsspezifikation ist vollständig, korrekt, enthält nur notwendige Anforderungen und ist gemäss dem Stand der Technik beschrieben.

Spannend war allerdings: Nur etwa **25% der Projekte wollten beides erreichen – gemeinsames Verständnis und hohe Spezifikationsqualität**. Weitere 25% fokussierten nur auf das gemeinsame Verständnis, wiederum 25% nur auf die Spezifikationsqualität, und die restlichen 25% kümmerten sich um keines von beiden. Überraschenderweise zeigten sich in der Projekterfolgsquote keine Unterschiede.

Diese Unterschiede spiegeln die Vielfalt der Projekte wider, in denen Requirements Engineering praktiziert wird:

- Am einen Ende des Spektrums: **öffentliche Ausschreibungen**, die von staatlichen Stellen gemäss WTO-Regeln durchgeführt werden. Hier werden Anforderungen in der Regel bereits vor Beginn des Dialogs mit potenziellen Anbietern definiert und veröffentlicht.
- Am anderen Ende: **agile Entwicklungsprojekte** mit Stakeholdern, die eng mit dem Team zusammenarbeiten. Hier werden Anforderungen oft **just-in-time** erarbeitet, wenn sie gebraucht werden, und umfangreiche Dokumentation wird vermieden.

1.2 Der Ermittlungszyklus – Prozess des Requirements Engineerings

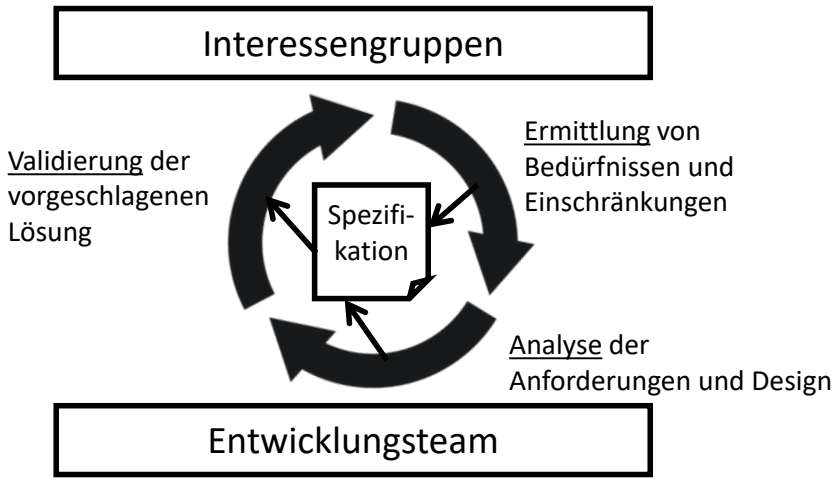


Abbildung 4: In Iterationen tauschen sich Interessengruppen wie Kund:innen und Benutzer:innen mit Mitgliedern des Entwicklungsteams wie Projektleitung, Architektur, Entwicklung und Testing aus. Der Requirements Engineer moderiert den Prozess.

Abbildung 4 zeigt die zentralen Aktivitäten des Requirements Engineering, mit denen ein gemeinsames Verständnis und qualitativ hochwertige Anforderungen erreicht werden sollen. In der dargestellten Reihenfolge bilden diese Aktivitäten einen **Inquiry Cycle**: einen Kreislauf zur Lösung der Stakeholder-Probleme mit einer passenden Lösung – dem Softwareprodukt.

Der Inquiry Cycle beschreibt, wie Stakeholder und Entwicklungsteam das Softwareprodukt **gemeinsam co-designen**. Die Spezifikation der Lösung unterstützt diesen Prozess und entsteht dabei als Nebenprodukt, indem die Ergebnisse der einzelnen Schritte dokumentiert werden.

Requirements Engineering ist ein **iterativer Prozess** mit den Schritten:

- **Elicitation (Erhebung, Ermittlung):** Der Kontext der Software wird erkundet, um Bedürfnisse und Rahmenbedingungen der Stakeholder zu erfassen.
- **Analyse:** Das Entwicklungsteam untersucht mögliche Lösungsoptionen, um Bedürfnisse und Rahmenbedingungen in Anforderungen zu überführen.
- **Validierung:** Umsetzungsvorschläge (z. B. Prototypen) werden mit Stakeholdern ausprobiert.
- **Spezifikation:** Ergebnisse aus Erhebung und Analyse – inklusive Prototypen – werden dokumentiert und dienen wiederum zur Validierung.

Requirements Engineering startet je nach Projekttyp unterschiedlich:

- **Kundenprojekt:** Hier beginnt RE auf der Seite der Stakeholder. Zuerst werden Vision, Bedürfnisse und Rahmenbedingungen erhoben, danach wird eine

vorgeschlagene Umsetzung validiert. Oft braucht es **zwei Iterationen**, um stabile Anforderungen zu erreichen.

- **Innovationsprojekt:** Hier beginnt RE auf der Seite des Entwicklungsteams. Es werden Vorschläge für ein mögliches Produkt entwickelt, die anhand von Bedürfnissen und Rahmenbedingungen validiert und angepasst werden. Häufig sind **mehrere Iterationen** nötig, bis Produkt und Markt gut zusammenpassen und die Anforderungen stabil sind.

Ein **gemeinsames Verständnis** zwischen Stakeholdern und Entwicklungsteam kann formell dokumentiert werden, z. B. durch eine Unterschrift oder schriftliche Bestätigung. Vorher findet üblicherweise ein Review oder eine formale Prüfung des Anforderungsdokuments statt.

Nach der Freigabe und während der Umsetzung werden die Anforderungen **verwaltet**: Sie steuern die Arbeit, erlauben kontrollierte Änderungen und bilden die Basis für System- und Abnahmetests.

4.4 Grundlegende Konzepte im Requirements Engineering

Grundlegende Konzepte im Requirements Engineering

Um mit **Gesichtspunkte (Blickwinkel, Sichtweisen, Perspektiven, Viewpoints)** arbeiten zu können, ist es wichtig, die Konzepte **Annahmen, Ideen, Bedürfnisse, Ziele und Anforderungen** zu verstehen:

- **Annahmen** entstehen durch Reflexion der Requirements Engineers.
- **Ideen** entstehen durch Kreativität des Entwicklungsteams, nach Möglichkeit zusammen mit den Interessengruppen.
- **Bedürfnisse** ergeben sich durch die Analyse der Gesichtspunkte.

Damit aus diesen Dingen **Ziele** oder **Anforderungen** werden, braucht es ein **„Handshake“ zwischen Stakeholdern und Entwicklungsteam**.

Gemäss dem **ISO/IEC/IEEE Standard 24765-2017** ist eine Anforderung:

1. eine Bedingung oder Fähigkeit, die eine*n* *Nutzer*in benötigt, um ein Problem zu lösen oder ein Ziel zu erreichen,
2. eine Bedingung oder Fähigkeit, die ein System oder eine Systemkomponente erfüllen muss, um einen Vertrag, Standard oder eine Spezifikation einzuhalten,

3. eine dokumentierte Darstellung einer Bedingung oder Fähigkeit gemäss (1) oder (2).

(1) und (2) verknüpfen Anforderungen einerseits mit den Stakeholdern, die sie benötigen, und andererseits mit dem System, das sie umsetzt. Zudem beziehen sich Anforderungen häufig auf Verträge oder Standards, die das Entwicklungsteam verpflichten. (3) betont, dass Anforderungen dokumentiert oder im gemeinsamen Verständnis festgehalten sein können.

Die folgende Tabelle charakterisiert die Konzepte im Vergleich:

Tabelle 1: Vergleich von Annahmen, Ideen, Bedürfnissen, Anforderungen und Zielen. Kreuze markieren verfügbare Fakten oder Beweise im Projekt.

	Wird von einer Interessengruppe benötigt	Wird vom Team geliefert	Ist ein Merkmal der Software	Ist eine Auswirkung der Software
Annahme				
Idee		X		
Bedürfnis	X			
Anforderung	X	X	X	
Ziel	X	X		X

Diese Charakterisierung ist hilfreich für die Arbeit im Requirements Engineering:

- Damit aus einer **Idee** eine **Anforderung oder ein Ziel** wird, muss ein Stakeholder gefunden werden, der sie benötigt.
- Damit aus einem **Bedürfnis** eine **Anforderung oder ein Ziel** wird, braucht es ein Entwicklungsteam oder einen Anbieter, der bereit ist, es umzusetzen.

Eine Anforderung entspricht in ihrer Charakterisierung einer **Funktion oder einem Feature** – also einer Menge von Anforderungen, die aus Sicht der Stakeholder zusammengehören.

Funktionale und Qualitätsanforderungen sowie Constraints

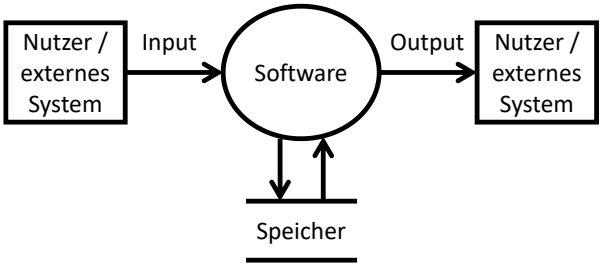


Abbildung 5: Definition einer Funktion (Notation: Datenflussdiagramm).

Funktionale Anforderungen beschreiben, wie die Software Eingaben verarbeitet, Daten speichert und wieder ausgibt sowie auf externe Systeme reagiert. (**Abbildung 5** zeigt dies schematisch als Datenflussdiagramm.)

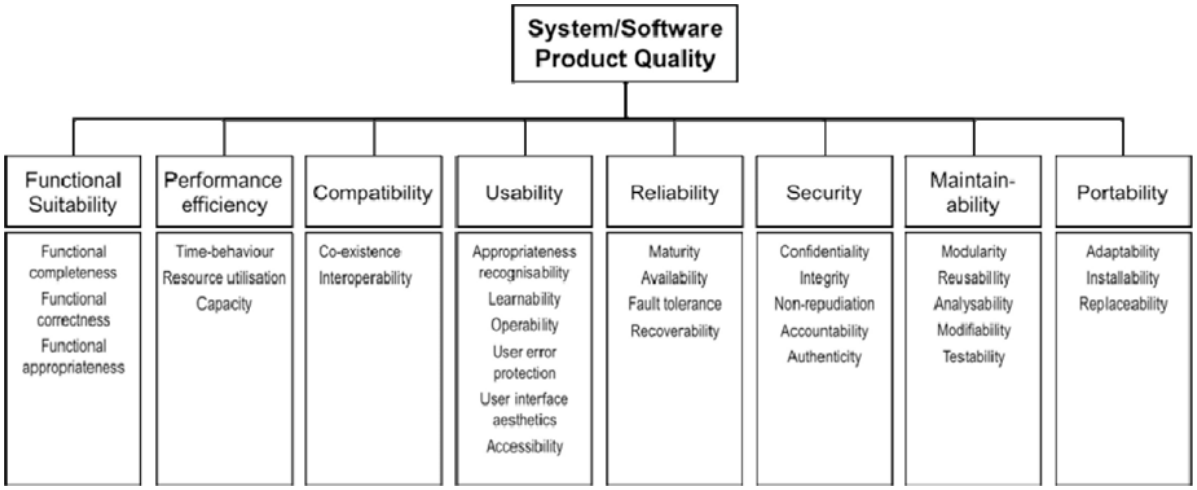


Abbildung 6: Häufige Qualitätseigenschaften gemäss ISO/IEC FIS 25010-2010.

Qualitätsanforderungen können sich unterscheiden nach Perspektive:

- *Nutzende:* Funktionale Eignung, Performance/Effizienz, Kompatibilität, Usability, Zuverlässigkeit, Sicherheit.
- *Entwicklungsteam:* Wartbarkeit (z. B. Weiterentwicklung oder Fehlerbehebung).
- *Betrieb/Deployment:* Portabilität (Installation und Anpassung in Zielumgebungen).

Mit dem Aufkommen von **IoT- und KI-Systemen** treten neue Qualitätsmerkmale hinzu, z. B. **Korrektheit, Vollständigkeit, Konsistenz, Vertrauenswürdigkeit, Aktualität, Präzision und Nachvollziehbarkeit von Daten und KI-Systemen**.

Rahmenbedingungen (Constraints, Randbedingungen) betreffen entweder die **Softwarearchitektur** oder das **Projektmanagement**. Sie sind für Nutzende nicht direkt sichtbar, schränken aber die Freiheit der Entwickelnden ein.

- Beispiele von Softwarebezogenen Rahmenbedingungen: Vorgaben zu Programmiersprache, Frameworks oder Zielplattform („Die Software muss in C# implementiert werden“ oder „Die Software muss auf Android 10.0 laufen“).
- Projektbezogene Rahmenbedingungen: Vorgehensmodell, Fristen, Budgetgrenzen.

4.5 Umsetzung in verschiedenen Softwareentwicklungsmodellen

Wann und wie Requirements Engineering durchgeführt wird und wie Anforderungen in einem Softwareprojekt spezifiziert werden, hängt stark vom verwendeten **Software Development Lifecycle Model (SDLM)** ab. Die wichtigsten Vorgehensmodelle sind:

- **Wasserfallmodell**
- **Agile Modelle** (z. B. Scrum)
- **Hybride Modelle** (z. B. Rational Unified Process, RUP)

Requirements Engineering im sequenziellen Wasserfallmodell

In einem Wasserfallprojekt werden die Anforderungen **zu Beginn des Projekts vollständig spezifiziert** und in einen ersten Softwareentwurf überführt [Royce 1970]. Während der Entwicklung, des Testens und der Einführung können sich Anforderungen zwar ändern, doch wird versucht, diese Änderungen durch **Reviews und ein striktes Change Management** möglichst gering zu halten.

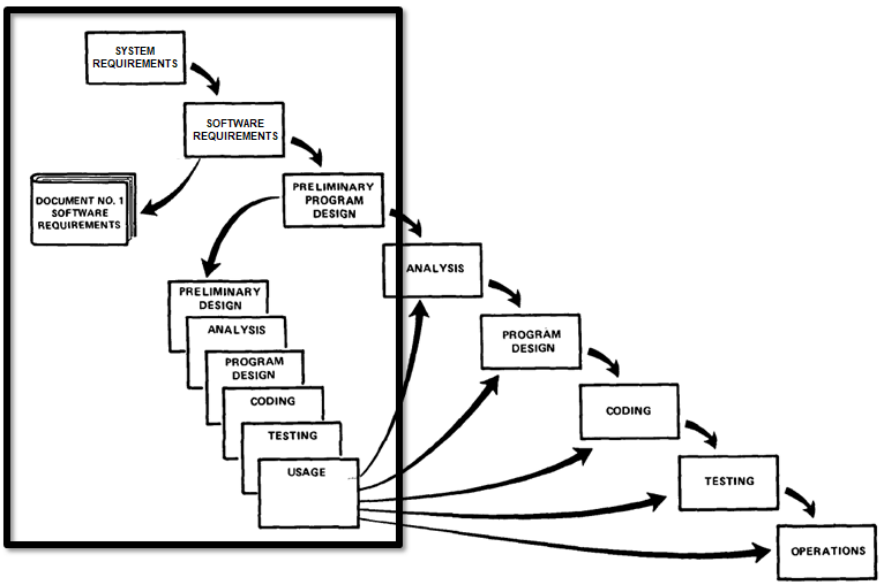


Abbildung 7: Requirements Engineering im Wasserfallmodell [Royce 1970].

Abbildung 7 zeigt das Requirements Engineering im Wasserfallmodell: Anforderungen werden am Anfang des Projekts spezifiziert und mit Hilfe eines Prototyps überprüft, ob sie geeignet sind und ob die vorgeschlagene Umsetzung passt.

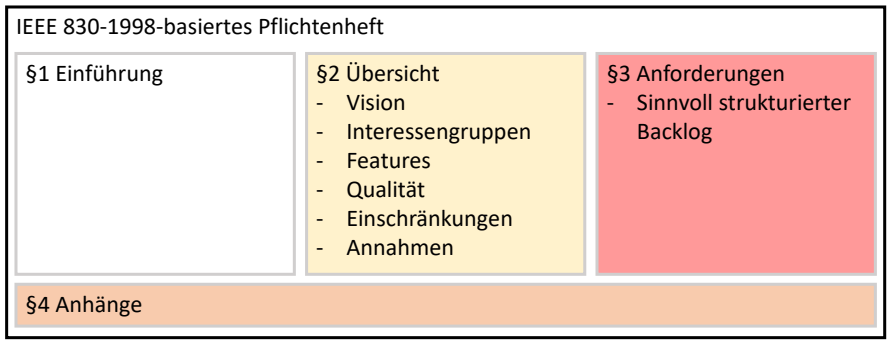


Abbildung 8: Struktur einer Software-Anforderungsspezifikation (SRS), dem Pflichtenheft, angelehnt an den Standard IEEE 830-1998.

Ein verbreiteter Standard ist die **IEEE 830-1998 Recommended Practice for Software Requirements Specifications (SRS)**. **Abbildung 8** zeigt schematisch die typische Struktur eines solchen Dokuments:

1. Erklärung des Dokuments
2. Zusammenfassung der Produktvision
3. Detaillierte Anforderungen (Backlog)
4. Anhang (z. B. Storyboards oder UI-Prototypen)

Ein neuerer Standard ist **ISO/IEC/IEEE 29148:2011**, der die Prinzipien von IEEE 830 erweitert und Requirements Engineering über die gesamte Wertschöpfungskette hinweg definiert – von der Marktanalyse bis hin zur Spezifikation einzelner Komponenten.

Requirements Engineering im iterativen agilen Projekt

In einem agilen Projekt werden Requirements Engineering und Implementierung **kontinuierlich und iterativ** während des ganzen Projekts wiederholt.

Die Anforderungen werden in einer sortierten Liste dokumentiert, im Scrum-Prozess **Product Backlog** genannt. Dieses wird in den **Sprint-Planungsmeetings** in detaillierte Aufgaben (Sprint Backlog) heruntergebrochen. Die Entwickler*innen arbeiten während eines Sprints an diesen Aufgaben und liefern am Ende einen **Produkt-Inkrement**. Dieses wird genutzt, um Anforderungen zu verfeinern und das Product Backlog zu aktualisieren. (**Abbildung 9**)

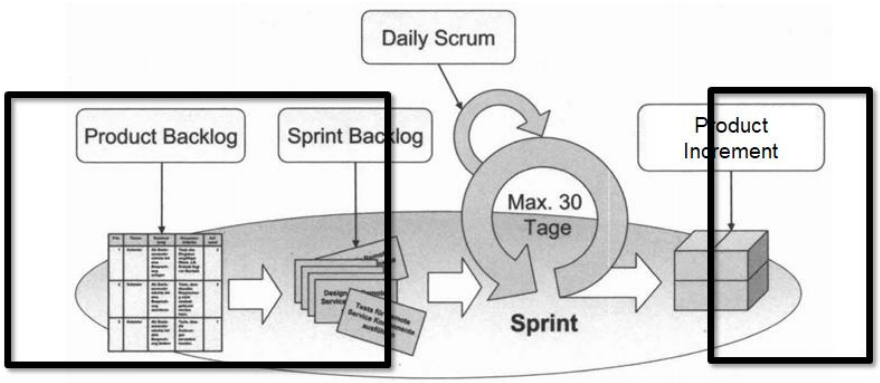


Abbildung 9: Requirements Engineering im agilen SCRUM-Prozess [Schwaber 2002; Pichler 2008].

Jedes Item im Product Backlog entspricht einer **User Story**.

- Die Liste ist priorisiert: oben stehen die dringendsten Anforderungen, unten diejenigen, die wahrscheinlich nie umgesetzt werden.
- Mit jeder Iteration wird die Liste ergänzt und neu sortiert.
- Strukturiert wird zusätzlich durch Gruppierung in **Features** (grob) und **Epics** (mittlere Granularität). (**Abbildung 10**)

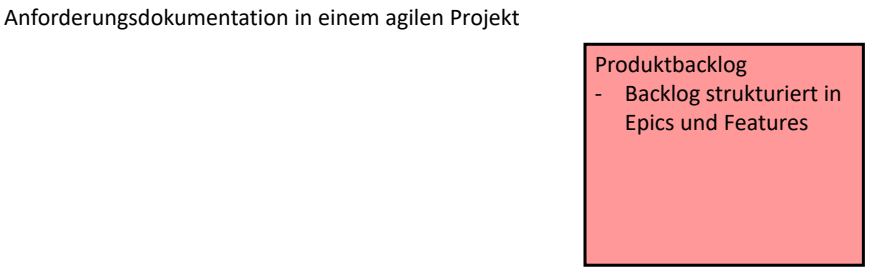


Abbildung 10: Struktur der Anforderungsdokumentation in der agilen Entwicklung.

Im agilen Vorgehen ist **kein fixes Dokument vorgesehen** – Anforderungen können in Tools oder sogar auf Post-its verwaltet werden. Dennoch können bei Bedarf zusätzliche Dokumente ergänzt werden; agile Ansätze schliessen die Übernahme von Ideen aus anderen Modellen nicht aus.

Requirements Engineering im hybriden Rational Unified Process (RUP)

Im RUP-Projekt konzentriert sich Requirements Engineering vor allem auf das erste Drittel des Projekts, wird aber auch danach laufend verfeinert.

RUP gilt als **hybrid**, weil es sequenzielle Projektphasen (Inception, Elaboration, Construction, Transition) mit **agilen Iterationen innerhalb der Phasen** kombiniert.

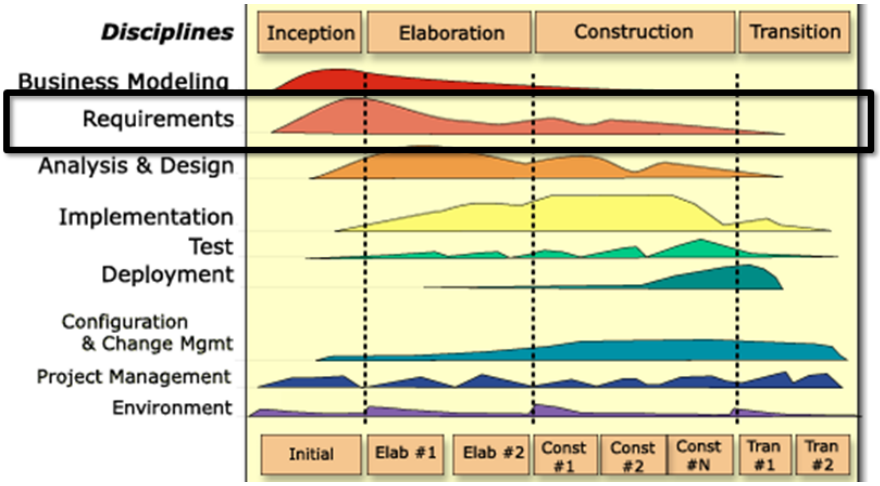


Abbildung 11: Requirements Engineering im hybriden Rational Unified Prozess [Kurchten 2004].

Die Anforderungen werden in mehreren Dokumenten erfasst:

- **Vision Document**: entspricht Abschnitt 2 „Overall Description“ der IEEE 830. Es entsteht aus den über längere Zeit gesammelten Stakeholder-Anfragen.
- **Use Case Documents**: detaillierte Spezifikationen einzelner Funktionen (ähnlich den User Stories in agilen Projekten). Zusammen bilden sie den Abschnitt 3 der IEEE 830.
- **Supplementary Requirements Document**: enthält Datenmodell und Qualitätsanforderungen (ebenfalls Teil von Abschnitt 3 der IEEE 830).
- **Storyboards**: zeigen, wie das UI-Prototyp die Arbeit oder Prozesse der Nutzenden unterstützt. (In IEEE 830 wären Storyboards im Anhang zu finden.)

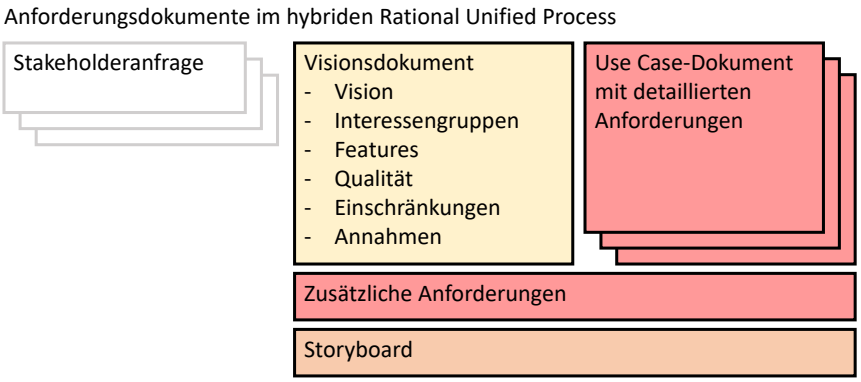


Abbildung 12: Struktur der Anforderungsdokumentation in einem RUP-basierten Projekt.

Referenzen

[Armour 2004] P. Armour, «Real Work, Necessary Friction, Optional Chaos», Communications of the ACM, 47(6):15-18, 2004.

[Clancy 1995] T. Clancy, «The Standish Group Report: Chaos», Technischer Bericht der Standish Group, 1995.

[Fricker 2012] S. Fricker, S. Schumacher, «Release Planning with Feature Trees: Industrial Case», 18th International Working Conference on Requirements Engineering (RefsQ 2012), Essen, Deutschland.

[Fricker 2015] S. Fricker, R. Grau, A. Zwingli, «Requirements Engineering: Best Practice», in S. Fricker, C. Thümmler, A. Gavras (Hrsg.): Requirements Engineering for Digital Health, Springer, 2015.

[Kniberg 2015] H. Kniberg, Scrum and XP from the Trenches, C4Media, 2015. ISBN 1329224272.

[Kruchten 2004] P. Kruchten, The Rational Unified Process: An Introduction (3. Aufl.), Addison-Wesley, 2004. ISBN 0-321-19770-4.

[Pichler 2008] R. Pichler, Scrum: Agiles Projektmanagement erfolgreich einsetzen, dpunkt.verlag, 2008. ISBN 978-3-89864-478-5.

[Pohl 2015] K. Pohl, C. Rupp, Requirements Engineering Fundamentals, Rocky Nook, 2. Aufl., 2015. ISBN 193753877X.

[Royce 1970] W. Royce, «Managing the Development of Large Software Systems», IEEE WESCON, 1970.

[Schwaber 2002] K. Schwaber, M. Beedle, Agile Software Development with Scrum, Pearson Education, 2002. ISBN 0-13-207489-3.

[Stapel 2012] K. Stapel, Informationsflusstheorie in der Softwareentwicklung, Dr. Hut, 2012. ISBN 3843906122.